Asynchronous JavaScript for API Handling 💻🌐

Asynchronous | Async/Await | JavaScript | Promises | API

- Introduction to Asynchronous JavaScript
  - Need for Asynchronous Programming
  - Common Use Cases
    - API Data Fetching
    - Form Submission (no refresh)
    - Background Synchronization
  - JavaScript's Async Handling
- JavaScript Event Loop & Async Execution Model
  - Call Stack
  - Event Loop
  - Microtasks vs. Macrotasks
  - Event Loop Execution Order Example
- Promises
  - What are Promises? (States)
  - fetch() and Promises Example
  - Chaining Promises (.then(), .catch())
  - Error Handling (.catch(), .finally())
- Handling API Errors & Timeouts
  - Common API Failures
  - Error Handling with Fetch & Async/Await
  - Implementing Timeouts (AbortController)
  - Error Handling and Retry Example
- Best Practices
  - Handle API Errors
  - Use async/await
  - Optimize API Requests
  - Use Promise.all()/allSettled()
  - Consider API Rate Limits
- Synchronous vs. Asynchronous Execution
  - Synchronous (Blocking UI)
  - Asynchronous (Non-blocking)
  - API Call Example (Blocking vs. Non-blocking)
- Callbacks (Legacy Approach)
  - What are Callbacks?
  - XMLHttpRequest Example
  - Problems with Callbacks (Callback Hell)
- Async/Await
  - What is Async/Await?
  - Async Function Example
  - Error Handling (try...catch)
  - Promises vs. Async/Await Comparison
- Practical Use Cases
  - Fetching & Displaying User Data
  - Form Submission with fetch()
  - Lazy Loading
  - Auto-suggestion Feature
  - Async API Calls in Vue.js (Optional)